

UNIVERSAL COMMAND AND CONTROL (C2) INTERFACE (UCI)

NORMALIZED INTERFACE SPECIFICATION (UNIS)



22 JANUARY 2026

Prepared By:
Open Architecture Collaborative Working Group (OACWG)

DISCLAIMER: The discussion of non-federal entities, methods, products, or services does not imply any endorsement by the United States Government, the Department of Defense, or the Department of the Air Force.

Distribution Statement A. Approved for public release: distribution is unlimited.

This page is intentionally left blank.

ABSTRACT

Universal Command and Control (C2) Interface (UCI) is a messaging standard supporting communication for mission-level C2. UCI provides a common message definition for performing mission operations and enables C2 coordination across weapon systems and weapon system elements such as sensors, vehicles, data products, and software. UCI is defined by the UCI Standard, which is comprised of the UCI Standard Document, the UCI Normalized Interface Specification (UNIS) (i.e., this document), the UCI Schema, and the UCI Schema Style and Design Specification. This document provides a set of requirements for normalized UCI message primitive interface behaviors.

The Universal Command and Control (C2) Interface (UCI) Schema Style and Design Specification (OAM-SPC-001) (USSDS) provides additional guidance on the structure of each UCI Message.

This page is intentionally left blank.

Document Number: OAC-SPC-002

Revision: A

REVISION RECORD

Revision	Date	Description
A	22 January 2026	Initial public release

See disclaimer and distribution A statements on cover page.

This page is intentionally left blank.

TABLE OF CONTENTS

1	Introduction.....	1
2	References.....	2
3	UCI Message Primitive Patterns Overview.....	3
4	Message Primitive Pattern Descriptions.....	4
4.1	Status-1.....	4
4.1.1	Status-1 Interaction Pattern.....	4
4.1.2	Status-1 Terminal State Behavior.....	5
4.2	Data-1.....	5
4.2.1	Data-1 Interaction Pattern.....	5
4.2.2	Data-1 Terminal State Behavior.....	6
4.3	DataRecord-1.....	6
4.3.1	DataRecord-1 Interaction Pattern.....	6
4.3.2	DataRecord-1 Terminal State Behavior.....	7
4.4	DataRequest-2.....	7
4.4.1	Data Request-2 Interaction Pattern.....	7
4.4.2	Data Request-2 Terminal State Behavior.....	8
4.5	ActionRequest-2.....	8
4.5.1	ActionRequest-2 Interaction Pattern.....	8
4.5.2	ActionRequest-2 Terminal State Behavior.....	8
4.6	Command-2.....	9
4.6.1	Command-2 Interaction Pattern.....	9
4.6.2	Command-2 Terminal State Behavior.....	9
5	Message Usage Descriptions.....	10
5.1	Data Record and List Management Interaction Patterns.....	10
5.1.1	DataRecordManagementRequest-3 Message Interaction Pattern.....	10
5.1.1.1	Examples of DataRecordManagementRequest-3 Message Interaction Pattern	11
5.1.1.2	Exemplars of DataRecordManagementRequest-3 Message Interaction Pattern	11
5.1.1.3	Template for DataRecordManagementRequest-3 Message Interaction Pattern	11
5.1.2	DataRecordListManagementRequest-3 Interaction Pattern.....	11
5.1.2.1	Examples of DataRecordListManagementRequest-3 Message Interaction Pattern.....	12
5.1.2.2	Exemplars of DataRecordListManagementRequest-3 Message Interaction Pattern.....	12
6	Acronyms and Abbreviations List.....	13

This page is intentionally left blank.

LIST OF FIGURES

Figure 4.1-1 Status-1 Message Interaction Pattern.....	5
Figure 4.2-1 Data-1 Message Interaction Pattern.....	6
Figure 4.3-1 DataRecord-1 Message Interaction Pattern.....	6
Figure 4.4-1 DataRequest-2 Message Interaction Pattern.....	7
Figure 4.5-1 ActionRequest-2 Message Interaction Pattern.....	8
Figure 4.6-1 Command-2 Message Interaction Pattern.....	9
Figure 5.1-1 DataRecordManagementRequest-3 Message Interaction Pattern.....	10
Figure 5.1-2 DataRecordListManagementRequest-3 Message Interaction Pattern.....	12

This page is intentionally left blank.

LIST OF TABLES

Table 2.0-1 Referenced Documents.....2
Table 3.0-1 Message Primitive Patterns.....3
Table 5.0-1 Acronyms.....13

This page is intentionally left blank.

1 INTRODUCTION

Universal Command and Control (C2) Interface (UCI) is a messaging standard supporting communication for mission-level C2. UCI defines a set of messages intended to standardize interfaces between software services for C2, and these messages have been defined and validated as fit for purpose across a variety of mission behaviors and heterogeneous mission system implementations. Standardization of suitable behaviors is accomplished through Normalization.

Normalization of message behaviors and patterns, within UCI, is a benefit to the community as it increases reuse, simplifies integration, increases the ability for the Adopting Program to interact and communicate with other UCI-enabled platforms, and reduces the cost of implementation.

2 REFERENCES

The documents listed in Table 2.0-1, Referenced Documents, are cited throughout this document. For dated references, only the revision cited applies. For undated references, the latest revision of the referenced document (including any addendums or corrigenda) applies.

Table 2.0-1 Referenced Documents

Document Number	Document Title	Revision	Date
OAC-STD-001	Universal Command and Control (C2) Interface (UCI) Standard	I	22 January 2026
OAC-STD-002	Universal Command and Control (C2) Interface (UCI) Schema	E	22 January 2026
OAC-SPC-001	Universal Command and Control (C2) Interface (UCI) Schema Style and Design Specification	E	22 January 2026

3 UCI MESSAGE PRIMITIVE PATTERNS OVERVIEW

UCI messages convey information between the different endpoints. Each message is assigned or mapped to one of six message primitive patterns. Messages are combined in specific ways called message primitive patterns which are designed to carry out different messaging tasks. There are six message primitive patterns: three single messages that provide data, data record(s), or status; and three paired messages requesting or commanding, and returning information. This section describes the message primitive patterns and describes how the messages in each primitive pattern interact. Refinements and additions to normalized interactions and behaviors will occur over time with a focus on proving greater interoperability amongst programs adopting UCI.

All six message primitive patterns are described in Table 3.0-1, Message Primitive Patterns. Note that all message primitive patterns are named using the format “<PatternName>-n”, where *n* is the number of messages utilized in the pattern.

Table 3.0-1 Message Primitive Patterns

Pattern Name	Description
<i>Status-1</i>	Consists of a single message that contains status.
<i>Data-1</i>	Consists of a single message that contains data.
<i>DataRecord-1</i>	Consists of a single message that is similar to <i>Data-1</i> but supports data record management.
<i>DataRequest-2</i>	Consists of two messages where the first message requests data (uses the suffix *DataRequest) and the second message is a response that will contain the status of the request (QUEUED, PROCESSING, COMPLETED, FAILED, REJECTED) and when the request is completed it may include the data (uses the suffix *DataRequestStatus)
<i>ActionRequest-2</i>	Consists of two messages where the first message requests an action (uses the suffix *Request) and the second message is a response that will contain the status of the request (QUEUED, PROCESSING, COMPLETED, FAILED, REJECTED) and when the action is completed it may include the requested data (uses the suffix *RequestStatus)
<i>Command-2</i>	Consists of two messages where the first message commands an action (uses the suffix *Command) and the second message is a response with the status (RECEIVED, ACCEPTED, REJECTED, CANCELED) of the command and when the command is ACCEPTED it may return a resulting activity (uses the suffix *CommandStatus)

The following subsections describe each message primitive pattern and how the messages in the pattern interact.

4 MESSAGE PRIMITIVE PATTERN DESCRIPTIONS

The six primitive patterns are described below. Each pattern has a brief description along with an interaction pattern for the interface between the participants in the message exchange. Additionally, a discussion of terminal state behavior is provided; specifically, the terminal state behavior refers to the interface between the participants in the message exchange. Care should be taken when reviewing the interaction pattern. Primitive patterns that use two different messages address the message exchange between participants. In these cases, the interaction pattern flow is affected by the value of a particular field in the response message. UCI does not require a specific transport, so there can be no assumption that messages come in any order or that there is guaranteed delivery. The interaction patterns take this into account. Also, there is no design implication assumed with respect to the state machine of the requestee/commandee other than once a terminal state is reported, the sequence should end. In other words, a requester/commander can deem the sequence complete once a terminal state is reported and ignore any further responses.

4.1 Status-1

The *Status-1* message primitive pattern consists of a single message sent from a producer to one or more consumers. This is very similar to the *Data-1* and *DataRecord-1* message primitive patterns. Typically, *Status-1* primitive messages have a suffix of “Status”. However, there are cases where *Status-1* messages do not have any specific nomenclature that can be used to identify it as a *Status-1*. Thus, a *Status-1* message may be named anything that does not match any other primitive identifier.

4.1.1 Status-1 Interaction Pattern

There are two roles in the *Status-1* message pattern: Producer and Consumer. The Producer is the UCI Component that initiates the sequence identified in Figure 4.1-1, Status-1 Message Interaction Pattern. The Consumer is the UCI Component that receives the message. Messages following the *Status-1* message primitive pattern do not include an acknowledgement from the consumer(s) as part of the message exchange. The *Status-1* pattern is akin to a “fire and forget” mechanism. As shown in Figure 4.1-1, *Status-1* messages take on two forms; the first is a message name ending with “Status” and the other is just a message name (that does not meet another naming convention for patterns denoted below).

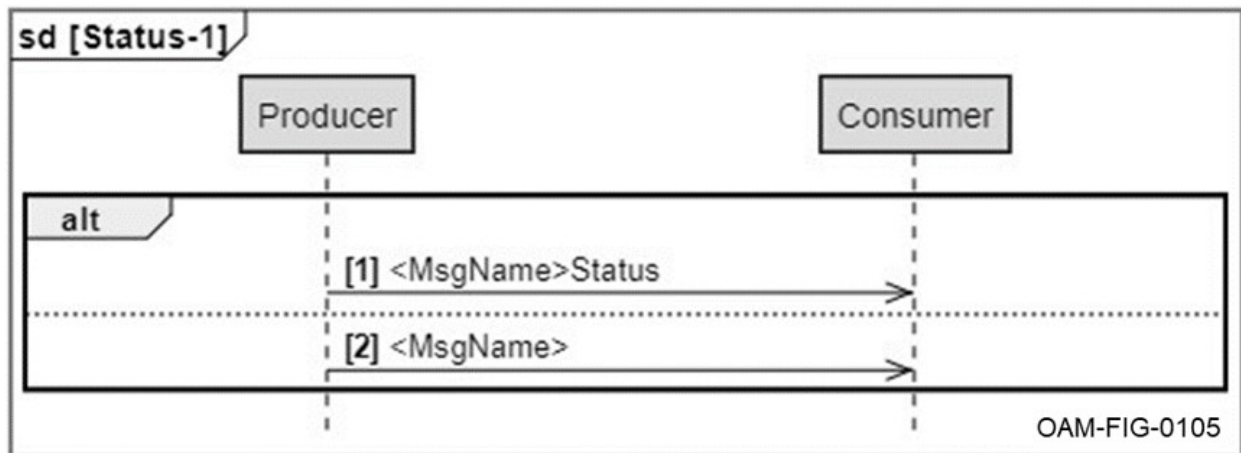


Figure 4.1-1 Status-1 Message Interaction Pattern

CERT UNIS-000076 [A UCI Component fulfilling a role identified in the Status-1 message interaction pattern shall follow the interaction pattern identified in the “Status-1 Message Interaction Pattern” figure.]

4.1.2 Status-1 Terminal State Behavior

When the *Status-1* message is sent, that is the terminal state. This is a “fire and forget” where once sent, the sequence is complete.

4.2 Data-1

The *Data-1* primitive pattern consists of a single message from a producer to one or more consumers. This is very similar to the *Status-1* and *DataRecord-1* message primitive patterns.

4.2.1 Data-1 Interaction Pattern

There are two roles in the *Data-1* message pattern: Producer and Consumer. The Producer is the UCI Component that initiates the sequence identified in Figure 4.2-1, Data-1 Message Interaction Pattern. The Consumer is the UCI Component that receives the message. Messages following the *Data-1* message primitive pattern do not include an acknowledgement from the consumer(s). The *Data-1* pattern is expected to be used as a “fire and forget” mechanism.



Figure 4.2-1 Data-1 Message Interaction Pattern

CERT UNIS-000081 [A UCI Component fulfilling a role identified in the Data-1 message interaction pattern shall follow the interaction pattern identified in the “Data-1 Message Interaction Pattern” figure.]

4.2.2 Data-1 Terminal State Behavior

When the *Data-1* message is sent, that is the terminal state. This is a “fire and forget” where once sent, the sequence is complete.

4.3 DataRecord-1

The *DataRecord-1* primitive pattern includes a single message from a producer to one or more consumers. This is very similar to the *Status-1* and *Data-1* message primitive patterns.

4.3.1 DataRecord-1 Interaction Pattern

There are two roles in the *DataRecord-1* message pattern: Producer and Consumer. The Producer is the UCI Component that initiates the sequence identified in Figure 4.3-1, DataRecord-1 Message Interaction Pattern. The Consumer is the UCI Component that receives the message. Messages following the *DataRecord-1* primitive pattern do not include an acknowledgement from the consumer(s). The DataRecord-1 pattern is expected to be used as a “fire and forget” mechanism. The two flows shown in this diagram indicate mutually exclusive paths; one with <MsgName>DataRecord sent with the optional DataRecordInstanceID included, and one without. In the case where the DataRecordInstanceID is omitted, the flow acts in the same manner as the *Data-1* message pattern.

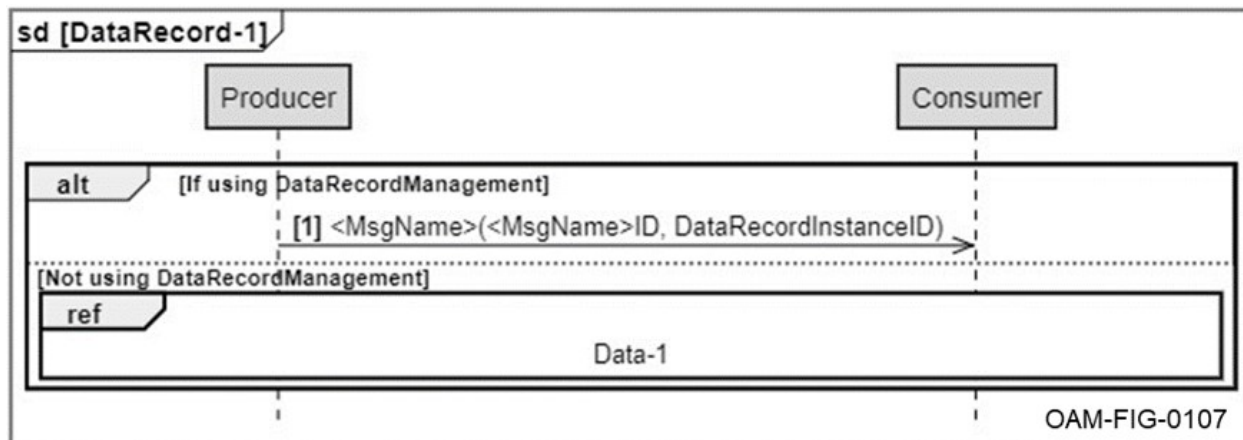


Figure 4.3-1 DataRecord-1 Message Interaction Pattern

CERT UNIS-000087 [A UCI Component fulfilling a role identified in the DataRecord-1 message interaction pattern shall follow the interaction pattern identified in the “DataRecord-1 Message Interaction Pattern” figure.]

4.3.2 DataRecord-1 Terminal State Behavior

When the *DataRecord-1* message is sent, that is the terminal state. This is a “fire and forget” where once sent, the sequence is complete.

4.4 DataRequest-2

The *DataRequest-2* primitive pattern is used to request data and to ensure that the requestee issues either an acknowledgement or the requested data per the response message. This primitive pattern includes two messages, a data request message from a requester to a requestee initiating the process, and a data request status message sent by the requestee providing a status or the requested data.

4.4.1 Data Request-2 Interaction Pattern

There are two roles in the *DataRequest-2* message pattern: Requester and Requestee. The Requester is the UCI Component that initiates the sequence identified in Figure 4.4-1, DataRequest-2 Message Interaction Pattern. The Requestee is the UCI Component that receives the request and responds. The *DataRequest-2* pattern is a send and response interaction. The Requester expects an acknowledgement from the Requestee.

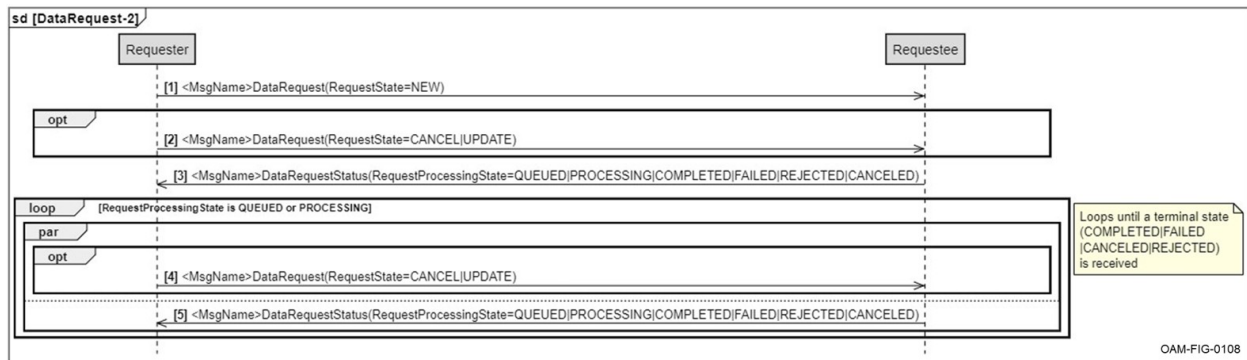


Figure 4.4-1 DataRequest-2 Message Interaction Pattern

CERT UNIS-000093 [A UCI Component fulfilling a role identified in the DataRequest-2 message interaction pattern shall follow the interaction pattern identified in the “DataRequest-2 Message Interaction Pattern” figure.]

4.4.2 Data Request-2 Terminal State Behavior

Once a request status message reporting a terminal state is received, the requester will ignore any further responses.

4.5 ActionRequest-2

The *ActionRequest-2* primitive pattern is used to request a process be performed and to ensure that the requestee issues either an acknowledgement or a status. This primitive pattern includes two messages, an action request message from a requester to a requestee initiating the process and an acknowledgement message sent by the requestee providing a status.

4.5.1 ActionRequest-2 Interaction Pattern

There are two roles in the *ActionRequest-2* message pattern: Requester and Requestee. The Requester is the UCI Component that initiates the sequence identified in Figure 4.5-1, ActionRequest-2 Message Interaction Pattern. The Requestee is the UCI Component that receives the request and responds. The *ActionRequest-2* pattern is a send and response interaction. The Requester expects an acknowledgement from the Requestee.

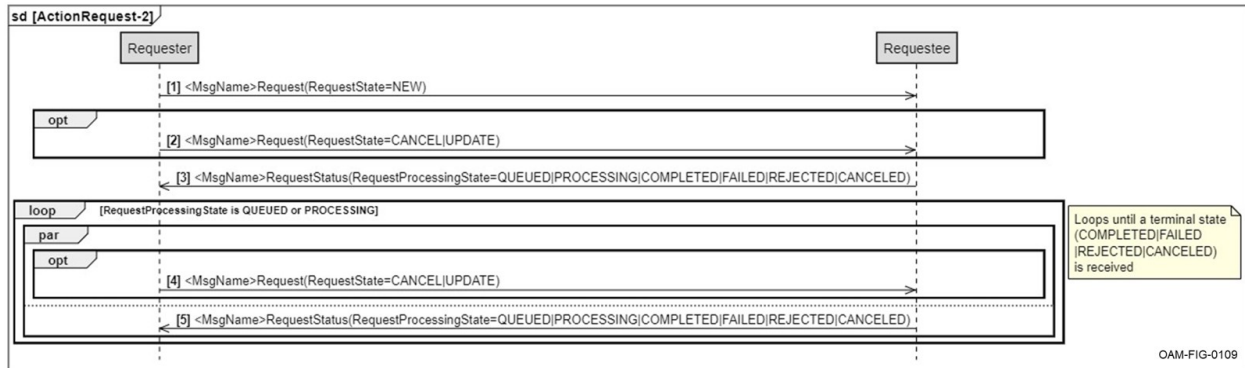


Figure 4.5-1 ActionRequest-2 Message Interaction Pattern

CERT UNIS-000099 [A UCI Component fulfilling a role identified in the ActionRequest-2 message interaction pattern shall follow the interaction pattern identified in the “ActionRequest-2 Message Interaction Pattern” figure.]

4.5.2 ActionRequest-2 Terminal State Behavior

Once a request status message reporting a terminal state is received, the requester will ignore any further responses.

4.6 Command-2

The *Command-2* primitive pattern is used to command the performance of an activity and ensures the commandee issues either an acknowledgement or a status. This primitive pattern includes two messages, a command message from a commander to a commandee initiating the activity, and an acknowledgement message sent by the commandee providing a status.

4.6.1 Command-2 Interaction Pattern

There are two roles in the *Command-2* message pattern: Commander and Commandee. The Commander is the UCI Component that initiates the sequence identified in Figure 4.6-1, Command-2 Message Interaction Pattern. The Commandee is the UCI Component that receives the command and responds. The *Command-2* primitive pattern is a send and response interaction. The Commander expects an acknowledgement from the Commandee.

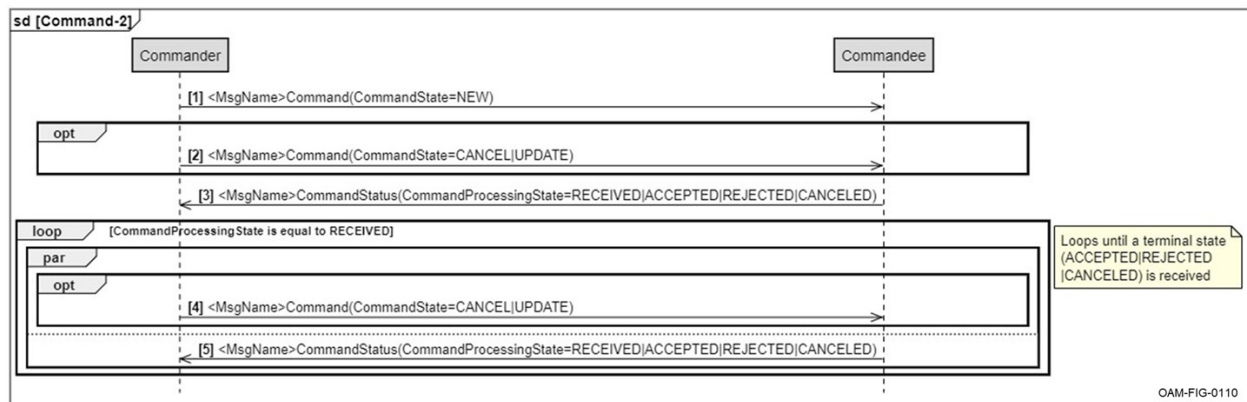


Figure 4.6-1 Command-2 Message Interaction Pattern

CERT UNIS-000105 [A UCI Component fulfilling a role identified in the Command-2 message interaction pattern shall follow the interaction pattern identified in the “Command-2 Message Interaction Pattern” figure.]

4.6.2 Command-2 Terminal State Behavior

Once a command status message reporting a terminal state is received, the commander will ignore any further responses.

5 MESSAGE USAGE DESCRIPTIONS

Message usage patterns are composed of a set of message primitive patterns and/or message group patterns. Those message usage patterns that have defined normalized behaviors are described in this section.

5.1 Data Record and List Management Interaction Patterns

5.1.1 DataRecordManagementRequest-3 Message Interaction Pattern

The DataRecordManagementRequest-3 message pattern involves three messages:

- **<MsgName>**
- **DataRecordManagementRequest**
- **DataRecordManagementRequestStatus**

The main message in this message pattern is an information message called **<MsgName>** that is published asynchronously and is defined as a *DataRecord-1* message primitive pattern.

There are two additional messages in this pattern called **DataRecordManagementRequest** and **DataRecordManagementRequestStatus** that are defined as an *ActionRequest-2* message primitive pattern.

Together, these three messages follow the pattern as shown in Figure 5.1-1, DataRecordManagementRequest-3 Message Interaction Pattern.



Figure 5.1-1 DataRecordManagementRequest-3 Message Interaction Pattern

CERT UNIS-000156 [A UCI Component fulfilling a role identified in the DataRecordManagementRequest-3 message interaction pattern shall follow the normalized behaviors identified in the “DataRecordManagementRequest-3 Message Interaction Pattern” figure.]

5.1.1.1 Examples of DataRecordManagementRequest-3 Message Interaction Pattern

There are many examples of this pattern that use the DataRecord-1 message primitive pattern along with an *ActionRequest-2* message primitive pattern in the UCI Schema today as shown in the UCI Schema Style and Design Specification Section 5.1.4, DataRecord-1 Message Primitive Pattern.

5.1.1.2 Exemplars of DataRecordManagementRequest-3 Message Interaction Pattern

<To Be Supplied (TBS) in a future release.>

5.1.1.3 Template for DataRecordManagementRequest-3 Message Interaction Pattern

<To Be Supplied (TBS) in a future release.>

5.1.2 DataRecordListManagementRequest-3 Interaction Pattern

The Data Record List Management pattern is an extension of the *DataRecord-1* pattern. It is designed to allow lists of data to be created and managed. Types created specifically for this pattern are described in the UCI Schema Style and Design Specification Section 4.3.1.5, Data Record List Types.

The *DataRecordListManagementRequest-3* message pattern involves three messages:

- **<MsgName>**
- **DataRecordListManagementRequest**
- **DataRecordListManagementRequestStatus**

The main message in this message group pattern is an information message called **<MsgName>** that is published asynchronously and is defined as a *DataRecord-1* message primitive.

There are two additional messages in this pattern called **DataRecordListManagementRequest** and **DataRecordListManagementRequestStatus** that are defined as an *ActionRequest-2* message primitive pattern.

Together, these three messages follow the pattern as shown in Figure 5.1-2, DataRecordListManagementRequest-3 Message Interaction Pattern.

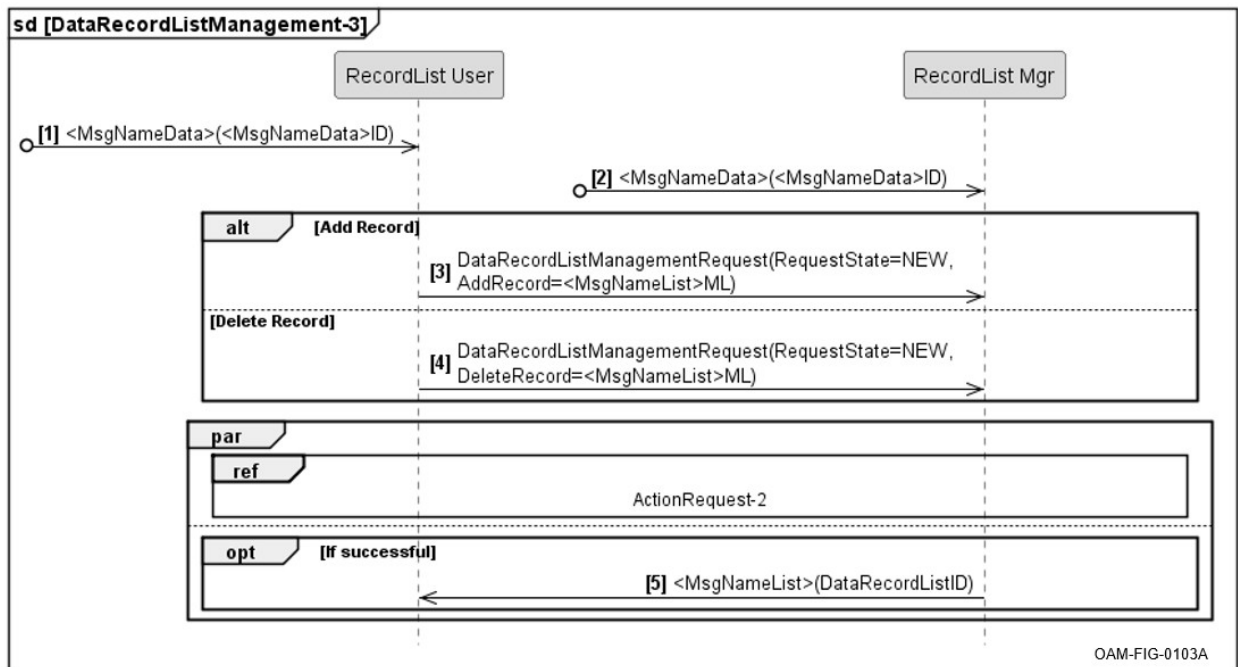


Figure 5.1-2 DataRecordListManagementRequest-3 Message Interaction Pattern

Messages 1 and 2 shown in Figure 5.1-2 are found messages. A found message is used when the originator of the message is undefined and unrepresented, but the lifelines must receive the message.

CERT UNIS-000190 [A UCI Component fulfilling a role identified in the DataRecordListManagementRequest-3 message interaction pattern shall follow the normalized behaviors identified in the “DataRecordListManagementRequest-3 Message Interaction Pattern” figure.]

5.1.2.1 Examples of DataRecordListManagementRequest-3 Message Interaction Pattern

Examples of this pattern include the **ForeignKeyMap** and **ForeignKeyPair** and the **MessageTransmissionList** and **MessageTransmissionListEntry** UCI messages.

5.1.2.2 Exemplars of DataRecordListManagementRequest-3 Message Interaction Pattern

<To Be Supplied (TBS) in a future release.>

6 ACRONYMS AND ABBREVIATIONS LIST

Table 5.0-1 Acronyms

Acronym	Definition
C2	Command and Control
CERT	Certification Requirement
OACWG	Open Architecture Collaborative Working Group
UCI	Universal C2 Interface
UNIS	UCI Normalized Interface Specification
USSDS	Universal Command and Control (C2) Interface (UCI) Schema Style and Design Specification